

Setup

Instructions

1. Work on a copy of this notebook: *File > Save a copy in Drive* (you will need a Google account).
2. (Optional) If you would like to do the deep learning component of this tutorial, turn on the GPU with Edit->Notebook settings->Hardware accelerator->GPU

```
!pip install -U pysr
```

Collecting pysr

Downloading pysr-1.5.10-py3-none-any.whl.metadata (54 kB)

Collecting click<9.0.0,>=7.0.0 (from pysr)

Using cached click-8.3.3-py3-none-any.whl.metadata (2.6 kB)

Collecting juliacall<0.9.27,>=0.9.24 (from pysr)

Downloading juliacall-0.9.26-py3-none-any.whl.metadata (4.5 kB)

Collecting numpy<3.0.0,>=1.13.0 (from pysr)

Using cached numpy-2.4.4-cp313-cp313-win_amd64.whl.metadata (6.6 kB)

Collecting pandas<4.0.0,>=0.21.0 (from pysr)

Using cached pandas-3.0.2-cp313-cp313-win_amd64.whl.metadata (19 kB)

Collecting scikit-learn<2.0.0,>=1.0.0 (from pysr)

Downloading scikit_learn-1.8.0-cp313-cp313-win_amd64.whl.metadata (11 kB)

Collecting sympy<2.0.0,>=1.0.0 (from pysr)

Using cached sympy-1.14.0-py3-none-any.whl.metadata (12 kB)

Collecting typing-extensions<5.0.0,>=4.0.0 (from pysr)

Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)

Requirement already satisfied: colorama in c:\users\knotp\documents\github\metodyoptymalizac\packages (from click<9.0.0,>=7.0.0->pysr) (0.4.6)

Collecting juliapkg<0.2,>=0.1.17 (from juliacall<0.9.27,>=0.9.24->pysr)

Downloading juliapkg-0.1.23-py3-none-any.whl.metadata (6.8 kB)

Collecting filelock<4.0,>=3.16 (from juliapkg<0.2,>=0.1.17->juliacall<0.9.27,>=0.9.24->pysr)

Using cached filelock-3.29.0-py3-none-any.whl.metadata (2.0 kB)

```

Collecting semver<4.0,>=3.0 (from juliapkg<0.2,>=0.1.17->juliacall<0.9.27,>=0.9.24->pysr)
  Downloading semver-3.0.4-py3-none-any.whl.metadata (6.8 kB)
Collecting tomli<3.0,>=2.0 (from juliapkg<0.2,>=0.1.17->juliacall<0.9.27,>=0.9.24->pysr)
  Downloading tomli-2.4.1-cp313-cp313-win_amd64.whl.metadata (10 kB)
Collecting tomlkit<0.15,>=0.13.3 (from juliapkg<0.2,>=0.1.17->juliacall<0.9.27,>=0.9.24->pysr)
  Downloading tomlkit-0.14.0-py3-none-any.whl.metadata (2.8 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\knotp\documents\github\metodyoptymalizacj
packages (from pandas<4.0.0,>=0.21.0->pysr) (2.9.0.post0)
Collecting tzdata (from pandas<4.0.0,>=0.21.0->pysr)
  Using cached tzdata-2026.2-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting scipy>=1.10.0 (from scikit-learn<2.0.0,>=1.0.0->pysr)
  Using cached scipy-1.17.1-cp313-cp313-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.3.0 (from scikit-learn<2.0.0,>=1.0.0->pysr)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.2.0 (from scikit-learn<2.0.0,>=1.0.0->pysr)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy<2.0.0,>=1.0.0->pysr)
  Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: six>=1.5 in c:\users\knotp\documents\github\metodyoptymalizacj
packages (from python-dateutil>=2.8.2->pandas<4.0.0,>=0.21.0->pysr) (1.17.0)
Downloading pysr-1.5.10-py3-none-any.whl (99 kB)
Using cached click-8.3.3-py3-none-any.whl (110 kB)
Downloading juliacall-0.9.26-py3-none-any.whl (12 kB)
Downloading juliapkg-0.1.23-py3-none-any.whl (21 kB)
Using cached filelock-3.29.0-py3-none-any.whl (39 kB)
Using cached numpy-2.4.4-cp313-cp313-win_amd64.whl (12.3 MB)
Using cached pandas-3.0.2-cp313-cp313-win_amd64.whl (9.7 MB)
Downloading scikit_learn-1.8.0-cp313-cp313-win_amd64.whl (8.0 MB)
----- 0.0/8.0 MB ? eta -:-:--
----- 1.3/8.0 MB 7.8 MB/s eta 0:00:01
----- 3.1/8.0 MB 8.6 MB/s eta 0:00:01
----- 5.0/8.0 MB 8.8 MB/s eta 0:00:01
----- 7.1/8.0 MB 8.9 MB/s eta 0:00:01
----- 8.0/8.0 MB 8.8 MB/s 0:00:00
Downloading semver-3.0.4-py3-none-any.whl (17 kB)
Using cached sympy-1.14.0-py3-none-any.whl (6.3 MB)
Using cached mpmath-1.3.0-py3-none-any.whl (536 kB)
Downloading tomli-2.4.1-cp313-cp313-win_amd64.whl (108 kB)
Downloading tomlkit-0.14.0-py3-none-any.whl (39 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Using cached scipy-1.17.1-cp313-cp313-win_amd64.whl (36.5 MB)
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)

```

```
Installing collected packages: mpmath, tzdata, typing-extensions, tomlkit, tomli, threadpool
learn, juliacall, pysr
```

3

[illegible]

[illegible]

```

----- 13/18 [pandas]
----- 13/18 [pandas]
----- 13/18 [pandas]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 15/18 [scikit-learn]
----- 17/18 [pysr]
----- 18/18 [pysr]

```

Successfully installed click-8.3.3 filelock-3.29.0 joblib-1.5.3 juliacall-0.9.26 juliapkg-0.1.23 mpmath-1.3.0 numpy-2.4.4 pandas-3.0.2 pysr-1.5.10 scikit-learn-1.8.0 scipy-1.17.1 semver-3.0.4 sympy-1.14.0 threadpoolctl-3.6.0 tomli-2.4.1 tomlkit-0.14.0 typing-extensions-4.15.0 tzdata-2026.2

[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip

Julia and Julia dependencies are installed at first import:

```
import pysr
```

```

[juliapkg] Found dependencies: c:\users\knotp\documents\github\metodyoptymalizacji\zad5\.venv\
packages\pysr\juliapkg.json
[juliapkg] Found dependencies: c:\users\knotp\documents\github\metodyoptymalizacji\zad5\.venv\
packages\juliacall\juliapkg.json
[juliapkg] Found dependencies: c:\users\knotp\documents\github\metodyoptymalizacji\zad5\.venv\
packages\juliapkg\juliapkg.json

```

```

[juliapkg] Locating Julia 1.10.3 - 1.11
[juliapkg] Querying Julia versions from https://julialang-s3.julialang.org/bin/versions.json
[juliapkg] WARNING: About to install Julia 1.11.9 to c:\Users\knotp\Documents\GitHub\MetodyO
[juliapkg] If you use juliapkg in more than one environment, you are likely to
[juliapkg] have Julia installed in multiple locations. It is recommended to
[juliapkg] install JuliaUp (https://github.com/JuliaLang/juliaup) or Julia
[juliapkg] (https://julialang.org/downloads) yourself.
[juliapkg] Downloading Julia from https://julialang-s3.julialang.org/bin/winnt/x64/1.11/julia
1.11.9-win64.zip
           downloaded 0.1 MB of 261.8 MB
           downloaded 47.2 MB of 261.8 MB
           downloaded 97.4 MB of 261.8 MB
           downloaded 147.9 MB of 261.8 MB
           downloaded 196.1 MB of 261.8 MB
           downloaded 246.9 MB of 261.8 MB
           download complete
[juliapkg] Verifying download
[juliapkg] Installing Julia 1.11.9 to c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\za
[juliapkg] Using Julia 1.11.9 at c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\zad5\..
[juliapkg] Using Julia project at c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\zad5\..
[juliapkg] Writing Project.toml:
           | [deps]
           | SymbolicRegression = "8254be44-1295-4e6a-a16d-46603ac705cb"
           | Serialization = "9e88b42a-f829-5b0c-bbe9-9e923198166b"
           | PythonCall = "6099a3de-0909-46bc-b1f4-468b9a2dfc0d"
           | OpenSSL_jll = "458c3c95-2e84-50aa-8efc-19380b2a3a95"
           |
           | [compat]
           | SymbolicRegression = "~1.11"
           | Serialization = "^1"
           | PythonCall = "=0.9.26"
           | OpenSSL_jll = "~3.0"
[juliapkg] Installing packages:
           | import Pkg
           | Pkg.Registry.update()
           | Pkg.add([
           |   Pkg.PackageSpec(name="SymbolicRegression", uuid="8254be44-
1295-4e6a-a16d-46603ac705cb"),
           |   Pkg.PackageSpec(name="Serialization", uuid="9e88b42a-f829-
5b0c-bbe9-9e923198166b"),
           |   Pkg.PackageSpec(name="PythonCall", uuid="6099a3de-0909-46bc-
b1f4-468b9a2dfc0d"),
           |   Pkg.PackageSpec(name="OpenSSL_jll", uuid="458c3c95-2e84-50aa-

```

```
8efc-19380b2a3a95"),
```

```
|  ])
```

```
| Pkg.resolve()
```

```
| Pkg.precompile()
```

Detected IPython. Loading juliacall extension. See <https://juliapy.github.io/PythonCall.jl/s>

Now, let's import everything else as well as the PySRRegressor:

```
import sympy
import numpy as np
from matplotlib import pyplot as plt
from pysr import PySRRegressor
from sklearn.model_selection import train_test_split
```

Simple PySR example:

First, let's learn a simple function

$$2.5382 \cos(x_3) + x_0^2 - 2$$

```
# Dataset
np.random.seed(0)
X = 2 * np.random.randn(100, 5)
y = 2.5382 * np.cos(X[:, 3]) + X[:, 0] ** 2 - 2
```

By default, we will set up 30 populations of expressions (which evolve independently except for migrations), use 4 threads, and use "best" for our model selection strategy:

```
default_pysr_params = dict(
    populations=30,
    model_selection="best",
)
```

PySR can run for arbitrarily long, and continue to find more and more accurate expressions. You can set the total number of cycles of evolution with `niterations`, although there are also a [few more ways](#) to stop execution.

This first execution will take a bit longer to startup, as the library is JIT-compiled. The next execution will be much faster.


```
# Learn equations
model = PySRRegressor(
    niterations=30,
    binary_operators=["+", "*"],
    unary_operators=["cos", "exp", "sin"],
    **default_pysr_params,
)

model.fit(X, y)
```

```
c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\zad5\.venv\Lib\site-
packages\pysr\sr.py:2811: UserWarning: Note: it looks like you are running in Jupyter. The p
warnings.warn(
Compiling Julia backend...
[ Info: Started!
[ Info: Final population:
[ Info: Results saved to:
```

Complexity	Loss	Score	Equation
1	3.360e+01	0.000e+00	y = 2.3552
3	6.128e+00	8.509e-01	y = x * x
5	3.045e+00	3.496e-01	y = (x * x) + -1.7558
7	3.030e+00	2.620e-03	y = ((x * x) * 0.97769) + -1.6641
8	1.118e+00	9.964e-01	y = (cos(x) + (x * x)) + -1.8519
9	9.515e-01	1.617e-01	y = ((x * x) + exp(cos(x))) + -3.1192
10	2.265e-13	2.907e+01	y = (x * x) + ((cos(x) * 2.5382) + -2)
12	2.132e-13	3.017e-02	y = ((x * x) + 0.16516) + ((cos(x) * 2.5382) + -
			2.1652)

- outputs\20260509_141842_yQBAsk\hall_of_fame.csv

model_selection	'best'
binary_operators	['+', '*']
unary_operators	['cos', 'exp', ...]
expression_spec	None
niterations	30
populations	30
population_size	27
max_evals	None

maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None
nested_constraints	None
elementwise_loss	None
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None
parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True
use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0
alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036
fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502
weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12

should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None
optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None
heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False
turbo	False
bumper	False
precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False
verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None
delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	None
extra_torch_mappings	None
extra_jax_mappings	None

denoise	False
select_k_features	None

We can print the model, which will print out all the discovered expressions:

```
model
```

model_selection	'best'
binary_operators	['+', '*']
unary_operators	['cos', 'exp', ...]
expression_spec	None
iterations	30
populations	30
population_size	27
max_evals	None
maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None
nested_constraints	None
elementwise_loss	None
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None
parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True
use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0
alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036

fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502
weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12
should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None
optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None
heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False
turbo	False
bumper	False
precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False

verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None
delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	None
extra_torch_mappings	None
extra_jax_mappings	None
denoise	False
select_k_features	None

We can also view the SymPy format of the best expression:

```
model.sympy()
```

$$x_0x_0 + \cos(x_3)2.5382 - 2.0$$

We can also view the SymPy of any other expression in the list, using the index of it in `model.equations_`.

```
model.sympy(2)
```

$$x_0x_0 - 1.755754$$

Output

`model.equations_` is a Pandas DataFrame. We can export the results in various ways:

```
model.latex()
```

```
'x_{0} x_{0} + \cos{\left(x_{3} \right)} 2.54 - 2.00'
```

These is also `model.sympy()`, `model.jax()`, `model.pytorch()`. All of these can take an index as input, to get the result for an arbitrary equation in the list.

We can also use `model.predict` for arbitrary equations, with the default equation being the one chosen by `model_selection`:

```
ypredict = model.predict(X)
ypredict_simpler = model.predict(X, 2)

print("Default selection MSE:", np.power(ypredict - y, 2).mean())
print("Manual selection MSE for index 2:", np.power(ypredict_simpler - y, 2).mean())
```

Default selection MSE: 0.0

Manual selection MSE for index 2: 3.045492013925884

Custom operators

A full list of operators is given here: <https://astroautomata.com/PySR/operators>, but we can also use any binary or unary operator in `julia`, or define our own as arbitrary functions.

Say that we want a command to do quartic powers:

$$y = x_0^4 - 2$$

```
y = X[:, 0] ** 4 - 2
```

We can do this by passing a string in `Julia` syntax.

We also define the operator in `sympy`, with `extra_sympy_mappings`, to enable its use in `predict`, and other export functions.

```
model = PySRRegressor(
    niterations=5,
    populations=40,
    binary_operators=["+", "*"],
    unary_operators=["cos", "exp", "sin", "quart(x) = x^4"],
    extra_sympy_mappings={"quart": lambda x: x**4},
)
model.fit(X, y)
```

```
c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\zad5\.venv\Lib\site-
packages\pysr\sr.py:2811: UserWarning: Note: it looks like you are running in Jupyter. The p
warnings.warn(
```

Complexity	Loss	Score	Equation
1	1.590e+04	0.000e+00	y = 46.897
2	4.000e+00	8.287e+00	y = quart(x)
4	9.015e-11	1.226e+01	y = quart(x) + -2
12	7.329e-11	2.589e-02	y = (quart(x) + -1.5451) + ((cos(exp(x)) * 2.709e-06) + ... -0.45493)

```
- outputs\20260509_142008_9Z8JQK\hall_of_fame.csv
```

```
[ Info: Started!
[ Info: Final population:
[ Info: Results saved to:
```

model_selection	'best'
binary_operators	['+', '*']
unary_operators	['cos', 'exp', ...]
expression_spec	None
niterations	5
populations	40
population_size	27
max_evals	None
maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None
nested_constraints	None
elementwise_loss	None
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None

parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True
use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0
alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036
fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502
weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12
should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None
optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None

heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False
turbo	False
bumper	False
precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False
verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None
delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	{'quart': <function <la...001A24E2F2200>>}
extra_torch_mappings	None
extra_jax_mappings	None
denoise	False
select_k_features	None

```
model.sympy()
```

$$x_0^4 - 1.9999995$$

Since `quart` is arguably more complex than the other operators, you can also give it a different complexity, using, e.g., `complexity_of_operators={"quart": 2}` to give it a complexity of 2 (instead of the default 2). You can also define custom complexities for variables and constants (`complexity_of_variables` and `complexity_of_constants`, respectively - both take a single number).

One can also add a binary operator, with, e.g., `"myoperator(x, y) = x^2 * y"`. All Julia operators that work on scalar 32-bit floating point values are available.

Make sure that any operator you add is valid over the real line. So, e.g., you will need to define `"mysqrt(x) = sqrt(abs(x))"` to enable it for negative numbers, or, simply have it return a very large number for bad inputs (to prevent negative input in a soft way): `"mysqrt(x::T) where {T} = (x >= 0) ? x : T(-1e9)"` (Julia syntax for a template function of input type T), which will make `mysqrt(x)` return -10^9 for negative x—hurting the loss of the equation.

Scoring

Using `model_selection="best"` selects the equation with the max score and prints it. But in practice it is best to look through all the equations manually, select an equation above some MSE threshold, and then use the score to select among that loss threshold.

Here, “score” is defined by:

$$\text{score} = -\log(\text{loss}_i / \text{loss}_{i-1}) / (\text{complexity}_i - \text{complexity}_{i-1})$$

This scoring is motivated by the common strategy of looking for drops in the loss-complexity curve.

From Schmidt & Lipson (2009) -

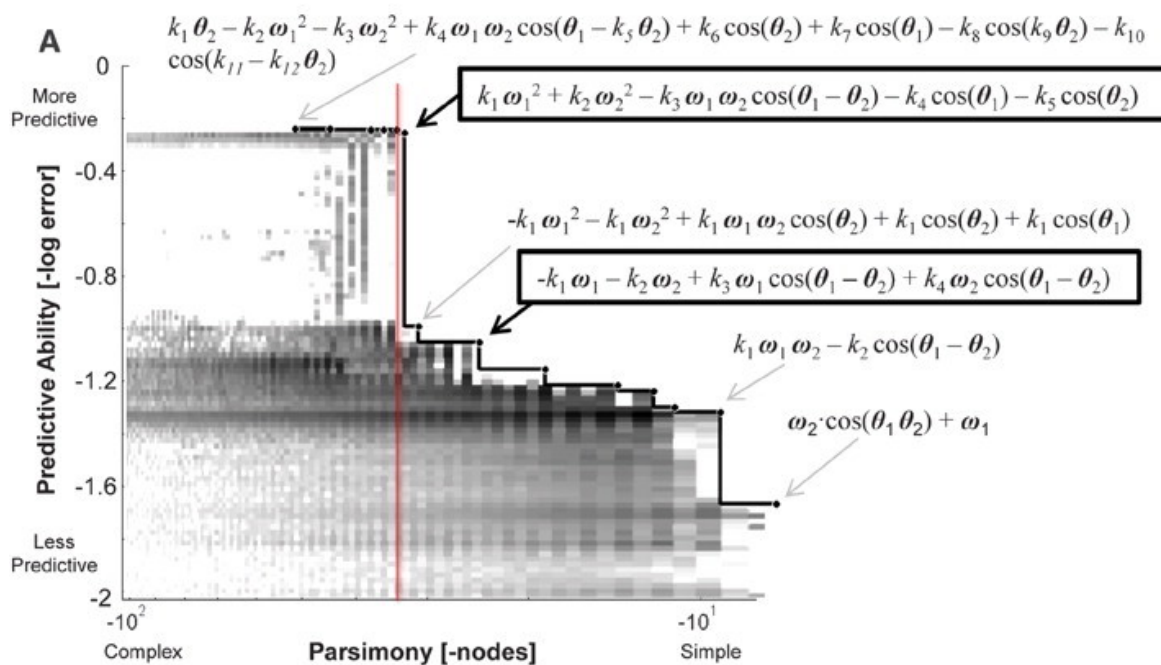


Figure 1: F4.large.jpg

Noise example

Here is an example with noise. Known Gaussian noise with σ between 0.1 and 5.0. We record samples of y :

$$\sigma \sim U(0.1, 5.0)$$

$$\epsilon \sim \mathcal{N}(0, \sigma^2)$$

$$y = 5 \cos(3.5x_0) - 1.3 + \epsilon.$$

We have 5 features, say. The weights change the loss function to be:

$$MSE = \sum [(y - f(x))^2 * w],$$

so in this example, we can set:

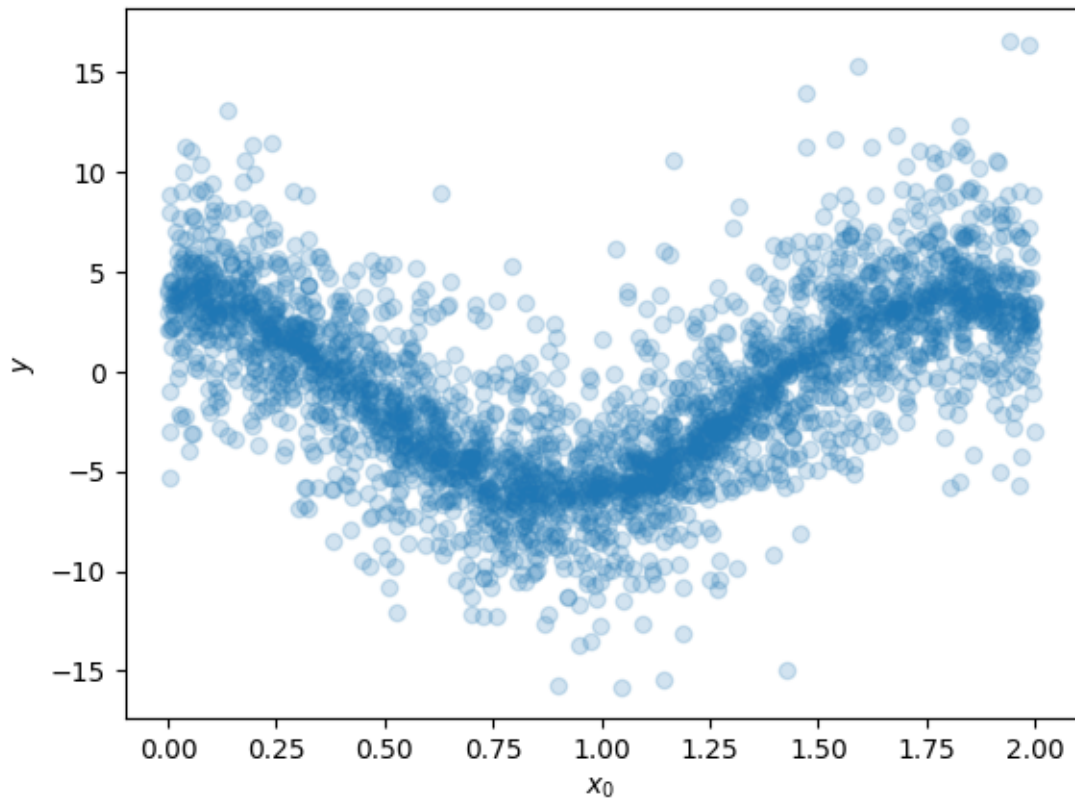
$$w = 1/\sigma^2.$$

```
np.random.seed(0)
N = 3000
upper_sigma = 5
X = 2 * np.random.rand(N, 5)
sigma = np.random.rand(N) * (5 - 0.1) + 0.1
eps = sigma * np.random.randn(N)
y = 5 * np.cos(3.5 * X[:, 0]) - 1.3 + eps
```

Let's look at this dataset:

```
plt.scatter(X[:, 0], y, alpha=0.2)
plt.xlabel("$x_0$")
plt.ylabel("$y$")
```

```
Text(0, 0.5, '$y$')
```



Define some weights to use:

```
weights = 1 / sigma**2
```

```
weights[:5]
```

```
array([0.25199296, 0.0775419 , 0.11515701, 0.1781281 , 0.0448131 ])
```

Let's run PySR again:

```
model = PySRRegressor(
    elementwise_loss="myloss(x, y, w) = w * abs(x - y)", # Custom loss function with weights
    niterations=20,
    populations=20, # Use more populations
    binary_operators=["+", "*"],
    unary_operators=["cos"],
)
model.fit(X, y, weights=weights)
```

```
c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\zad5\.venv\Lib\site-
packages\pysr\sr.py:2811: UserWarning: Note: it looks like you are running in Jupyter. The p
warnings.warn(
[ Info: Started!
```

```
Expressions evaluated per second: 6.750e+04
Progress: 376 / 400 total iterations (94.000%)
```

Complexity	Loss	Score	Equation
1	3.304e+00	0.000e+00	$y = -1.1849$
3	3.183e+00	1.860e-02	$y = x + -2.0869$
4	2.743e+00	1.488e-01	$y = \cos(x * -3.3337)$
6	1.089e+00	4.618e-01	$y = \cos(x * 3.2912) * 4.992$
8	3.126e-01	6.242e-01	$y = (\cos(x * 3.4967) + -0.2565) * 5.0317$
10	3.126e-01	5.782e-05	$y = ((\cos(x * 3.4951) + -0.10836) * 5.0216) + -$
0.74432			
14	3.115e-01	8.387e-04	$y = (\cos(\cos(\cos(x) * 0.063799) * (x * 3.4989)) * 5.0227.$
			$) + -1.2797$

Press 'q' and then <enter> to stop execution early.

Complexity	Loss	Score	Equation
1	3.304e+00	0.000e+00	$y = -1.1849$
3	3.183e+00	1.860e-02	$y = x + -2.0869$
4	2.743e+00	1.488e-01	$y = \cos(x * -3.3337)$
6	1.089e+00	4.618e-01	$y = \cos(x * 3.2912) * 4.992$
8	3.126e-01	6.242e-01	$y = (\cos(x * 3.4958) * 5.0322) + -1.2867$
10	3.126e-01	4.858e-05	$y = ((\cos(x * 3.4951) + -0.10836) * 5.0216) + -$
0.74432			
14	3.115e-01	8.387e-04	$y = (\cos(\cos(\cos(x) * 0.063799) * (x * 3.4989)) * 5.0227.$
			$) + -1.2797$

- outputs\20260509_142045_yAu4nV\hall_of_fame.csv

[Info: Final population:

[Info: Results saved to:

model_selection

'best'

binary_operators	['+', '*']
unary_operators	['cos']
expression_spec	None
niterations	20
populations	20
population_size	27
max_evals	None
maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None
nested_constraints	None
elementwise_loss	'myloss(x, y, w) = w * abs(x - y)'
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None
parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True
use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0
alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036
fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502

weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12
should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None
optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None
heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False
turbo	False
bumper	False
precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False
verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None

delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	None
extra_torch_mappings	None
extra_jax_mappings	None
denoise	False
select_k_features	None

Let's see if we get similar results to the true equation

`model`

model_selection	'best'
binary_operators	['+', '*']
unary_operators	['cos']
expression_spec	None
niterations	20
populations	20
population_size	27
max_evals	None
maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None
nested_constraints	None
elementwise_loss	'myloss(x, y, w) = w * abs(x - y)'
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None
parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True

use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0
alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036
fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502
weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12
should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None
optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None
heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False

turbo	False
bumper	False
precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False
verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None
delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	None
extra_torch_mappings	None
extra_jax_mappings	None
denoise	False
select_k_features	None

We can also filter all equations up to 2x the most accurate equation, then select the best score from that list:

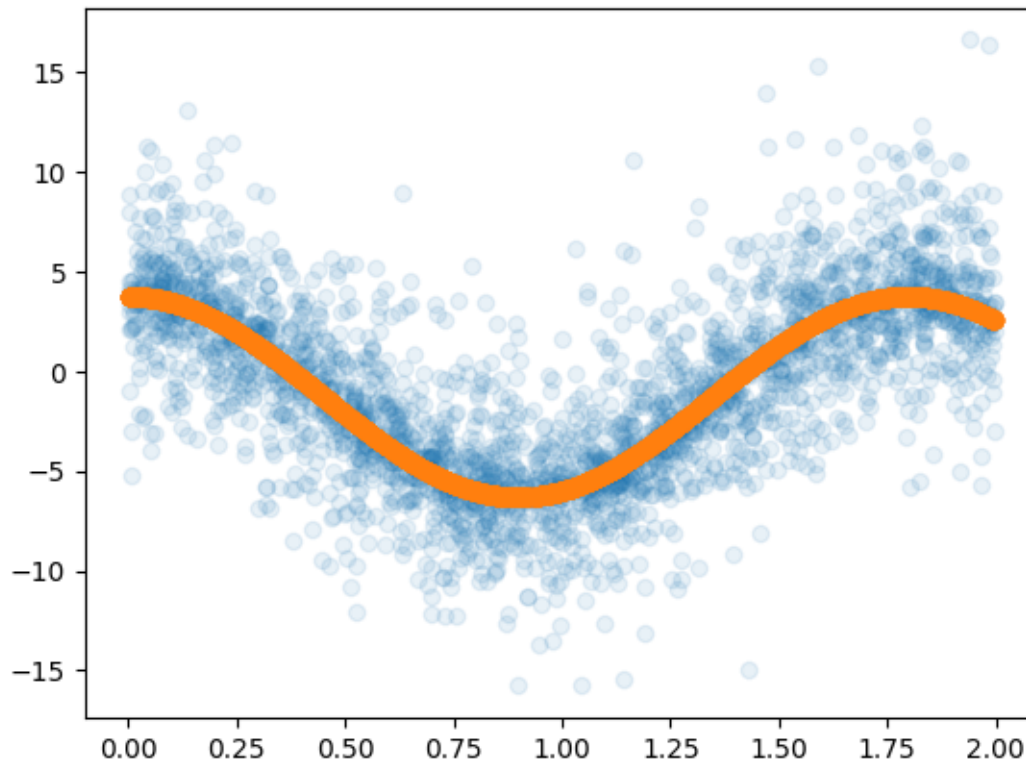
```
best_idx = model.equations_.query(
    f"loss < {2 * model.equations_.loss.min()}")
).score.idxmax()
model.sympy(best_idx)
```

$\cos(x_0 \cdot 3.4958313)5.0321765 - 1.2867261$

We can also use `denoise=True`, which will run the input through a Gaussian process to denoise the dataset, before fitting on it.

Let's look at the fit:

```
plt.scatter(X[:, 0], y, alpha=0.1)
y_prediction = model.predict(X, index=best_idx)
plt.scatter(X[:, 0], y_prediction)
```



Multiple outputs

For multiple outputs, multiple equations are returned:

```
X = 2 * np.random.randn(100, 5)
y = 1 / X[:, [0, 1, 2]]
```

```
model = PySRRegressor(
    binary_operators=["+", "*"],
    unary_operators=["inv(x) = 1/x"],
    extra_sympy_mappings={"inv": lambda x: 1 / x},
)
model.fit(X, y)
```

```
c:\Users\knotp\Documents\GitHub\MetodyOptymalizacji\zad5\.venv\Lib\site-
packages\pysr\sr.py:2811: UserWarning: Note: it looks like you are running in Jupyter. The p
warnings.warn(
[ Info: Started!
```

Expressions evaluated per second: 6.530e+05
Progress: 2989 / 9300 total iterations (32.140%)

Best equations for output 1

Complexity	Loss	Score	Equation
1	4.969e+01	0.000e+00	$y = -1.3212$
2	3.991e-14	3.476e+01	$y = \text{inv}(x)$
6	3.991e-14	5.563e-05	$y = (x * 4.9903e-09) + \text{inv}(x)$
7	3.139e-14	2.401e-01	$y = \text{inv}((\text{inv}(x) * 2.2184e-10) + x)$
9	3.082e-14	9.138e-03	$y = \text{inv}((x * (\text{inv}(x) * 1.398e-10)) + x)$
11	2.172e-14	1.749e-01	$y = \text{inv}((\text{inv}((1.4777 * x) + x) * -7.2125e-10) + x)$
13	1.562e-14	1.649e-01	$y = (\text{inv}(x * (x + 0.090373)) * -3.3928e-09) + (\text{inv}(x) * \dots + 1.4789e-08)$
14	1.276e-14	2.020e-01	$y = ((\text{inv}((x + 0.090373) * x) + \text{inv}(x)) * -2.5837e-09) + \dots + \text{inv}(x)$
18	1.275e-14	1.740e-04	$y = \text{inv}(x) + (((\text{inv}((x + 0.090373) * x) + (\text{inv}(x) + \dots 0.57604)) * -2.5837e-09) + -2.5837e-09)$
23	1.275e-14	1.045e-04	$y = (((\text{inv}((x + 0.090373) * x) + (((\text{inv}(x) * x) * -0.8282) + \text{inv}(x))) * -2.5837e-09) + -2.5837e-09) + \text{inv}(x) \dots$
29	1.263e-14	1.546e-03	$y = (\text{inv}(x) + 6.657e-10) + ((\text{inv}((x + 0.076032) * (\text{inv}((x) + (x * 0.056105))) + \text{inv}((x * (x * 4.1037)) * \dots 0.41825) + x)) * 4.2383e-09)$

Best equations for output 2

Complexity	Loss	Score	Equation
------------	------	-------	----------

1	6.356e+02	0.000e+00	y = -0.22241
2	2.336e-12	3.324e+01	y = inv(x)
6	2.336e-12	2.131e-06	y = inv(x) + (x * -4.92e-09)
7	1.358e-14	5.148e+00	y = inv(x) + (inv(x) * -4.0195e-08)
9	7.609e-15	2.896e-01	y = (inv(x + 0.0063144) * 1.4035e-08) + inv(x)
11	7.605e-15	2.919e-04	y = inv(x) + ((inv(x + 0.0019782) + 0.054094) * - 2.685... 2e-08)
13	7.538e-15	4.399e-03	y = inv(x) + (inv(x * (x + (x + 2.8532))) * - 5.0997e... -09)
17	7.467e-15	2.368e-03	y = inv(x) + (inv(x * (x + (x + ((x * x) + 2.8532)...))) * -5.0997e-09)
19	7.396e-15	4.780e-03	y = (inv(x * (((x + (x * (x * x))) + 2.8532) + x))... * -5.0997e-09) + inv(x)
21	6.926e-15	3.288e-02	y = (3.4766e-07 * inv(((x + ((x * (x + 2.7399)) * x)...) + (x + 2.8532)) * -3.7827)) + inv(x)
23	6.801e-15	9.059e-03	y = (inv((x + (((x * (x + (2.7399 + x))) * x) + (x... + 2.8532))) * -3.7827) * 3.4766e- 07) + inv(x)
25	5.762e-15	8.290e-02	y = inv(x) + (inv(x * ((x + (((x + (x * - 0.25341)) ... * ((x + 0.38564) * x)) + x)) + 2.8655)) * 3.3153e- 09)
29	5.629e-15	5.848e-03	y = (inv((x + (((x * (x + (((x + 0.22045) + x) + x...))) * x) + (x + 2.8532))) * -3.7827) * 3.4766e- 07) + (i... nv(x) + 7.2953e-09)

Best equations for output 3

Complexity	Loss	Score	Equation
1	1.910e+04	0.000e+00	y = 13.156
2	1.490e-10	3.248e+01	y = inv(x)
4	6.788e-15	4.998e+00	y = inv(x + 4.7591e-11)
9	6.779e-15	2.619e-04	y = inv(x + 8.5559e-11) + (inv(x) * 8.5559e- 11)
11	3.839e-15	2.843e-01	y = inv(x + 7.9835e-11) + (inv(x + - 0.23559) * 2.66e-0... 8)
13	2.240e-15	2.693e-01	y = inv(x + 5.3961e-11) + (4.0376e-08 * inv(x + (x + ... -0.40427)))

Press 'q' and then <enter> to stop execution early.

Expressions evaluated per second: 6.380e+05
Progress: 5656 / 9300 total iterations (60.817%)

Best equations for output 1

Complexity	Loss	Score	Equation
1	4.969e+01	0.000e+00	$y = -1.3212$
2	3.991e-14	3.476e+01	$y = \text{inv}(x)$
6	3.991e-14	5.563e-05	$y = (x * 4.9903e-09) + \text{inv}(x)$
7	3.139e-14	2.401e-01	$y = \text{inv}((\text{inv}(x) * 2.2184e-10) + x)$
9	3.082e-14	9.138e-03	$y = \text{inv}((x * (\text{inv}(x) * 1.398e-10)) + x)$
11	1.561e-14	3.400e-01	$y = (\text{inv}(x * (0.090373 + x))) * -3.3928e-09) + \text{inv}(x)$
13	1.550e-14	3.783e-03	$y = ((\text{inv}((x + 0.090373) * x) + -3.2198) * -2.5837e-09...)$ $\quad \quad \quad) + \text{inv}(x)$
14	1.276e-14	1.941e-01	$y = ((\text{inv}((x + 0.090373) * x) + \text{inv}(x)) * -2.5837e-09...)$ $\quad \quad \quad) + \text{inv}(x)$
16	6.706e-15	3.218e-01	$y = ((\text{inv}(x + 1.6718) + \text{inv}(x * (x + 0.090373)))) * -3...)$ $\quad \quad \quad .2637e-09) + \text{inv}(x)$
18	6.586e-15	9.021e-03	$y = ((\text{inv}(x + 1.6718) + (-3.8179 + \text{inv}((x + 0.090373) ... * x))) * -2.5837e-09) + \text{inv}(x)$
19	3.819e-15	5.449e-01	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x) + \text{inv}((x + 0.090373) ... * x))) * -2.5837e-09) + \text{inv}(x)$
21	3.677e-15	1.896e-02	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x * 0.87396) + \text{inv}((0.090... 373 + x) * x))) * -2.5837e-09) + \text{inv}(x)$
23	3.668e-15	1.209e-03	$y = (((0.75048 + (\text{inv}(x + 1.6718) + x)) + (\text{inv}(x) + i... nv(x * (x + 0.090373)))) * -2.5534e-$ $09) + \text{inv}(x)$
25	3.610e-15	7.932e-03	$y = \text{inv}(x) + (((\text{inv}(x + (x * x)) + \text{inv}(x + 1.6718)...) + \text{inv}(x * (x + 0.090373))) + x) * -2.5534e-09)$
27	3.599e-15	1.540e-03	$y = \text{inv}(x) + ((\text{inv}(x + (x * x)) + ((\text{inv}(x + 1.6718)... + \text{inv}(x * (x + 0.090373)))) + (x + -0.89466))) * -2.55...)$ $\quad \quad \quad 34e-09)$
28	3.570e-15	8.052e-03	$y = (((x + \text{inv}(x + 1.6718)) + (\text{inv}(x) + (\text{inv}(x * (x ...$

2.5534e-09)...
 $+ 0.090373)) + \text{inv}(x + (1.6718 + x)))) * -$
 $+ \text{inv}(x)$

Best equations for output 2

Complexity	Loss	Score	Equation
1	6.356e+02	0.000e+00	$y = -0.22241$
2	2.336e-12	3.324e+01	$y = \text{inv}(x)$
6	2.336e-12	2.131e-06	$y = \text{inv}(x) + (x * -4.92e-09)$
7	1.358e-14	5.148e+00	$y = \text{inv}(x) + (\text{inv}(x) * -4.0195e-08)$
9	7.609e-15	2.896e-01	$y = (\text{inv}(x + 0.0063144) * 1.4035e-08) + \text{inv}(x)$
11	7.598e-15	7.300e-04	$y = \text{inv}(x) + ((\text{inv}(x + 0.0039564) + -$ $0.080935) * -2.68...$ $52e-08)$
13	7.538e-15	3.961e-03	$y = \text{inv}(x) + (\text{inv}(x * (x + (x + 2.8532)))) * -$ $5.0997e...$ $-09)$
14	5.336e-15	3.456e-01	$y = \text{inv}(x) + ((\text{inv}(x + 0.0047919) + \text{inv}(x + -$ $0.21554)...$ $) * -7.432e-09)$
16	5.331e-15	4.163e-04	$y = \text{inv}(x) + (((\text{inv}(x + -0.21554) + 0.62131) + \text{inv}(x ...$ $+ 0.0047919)) * -7.432e-09)$
24	4.554e-15	1.969e-02	$y = (\text{inv}(((x + x) + ((\text{inv}((x + 0.97401) + x) + 2.853...$ $2) * (x * x))) + 2.8532) * -7.2074e-$ $08) + \text{inv}(x)$
25	3.666e-15	2.169e-01	$y = (\text{inv}(((x + ((x * (\text{inv}(\text{inv}(x + -$ $3.5763) + x) + 2....$ $8532)) * x)) + x) + 2.8532) * -$ $7.2074e-08) + \text{inv}(x)$

Best equations for output 3

Complexity	Loss	Score	Equation
1	1.910e+04	0.000e+00	$y = 13.156$
2	1.490e-10	3.248e+01	$y = \text{inv}(x)$
4	6.788e-15	4.998e+00	$y = \text{inv}(x + 4.7591e-11)$
9	6.779e-15	2.619e-04	$y = \text{inv}(x + 8.5559e-11) + (\text{inv}(x) * 8.5559e-$ $11)$
11	2.809e-15	4.405e-01	$y = \text{inv}(x + 7.9835e-11) + (\text{inv}(x + -$ $0.19597) * 2.66e-0...$

$$y = \frac{2.098e-15}{\text{inv}(x + 4.6222e-11) + (\text{inv}((x + -0.39619) + x) * \dots)} + \frac{1.458e-01}{4.5114e-08}$$

Press 'q' and then <enter> to stop execution early.

Expressions evaluated per second: 6.260e+05
Progress: 8279 / 9300 total iterations (89.022%)

Best equations for output 1

Complexity	Loss	Score	Equation
1	4.969e+01	0.000e+00	$y = -1.3212$
2	3.991e-14	3.476e+01	$y = \text{inv}(x)$
6	3.991e-14	5.563e-05	$y = (x * 4.9903e-09) + \text{inv}(x)$
7	3.139e-14	2.401e-01	$y = \text{inv}((\text{inv}(x) * 2.2184e-10) + x)$
9	2.172e-14	1.840e-01	$y = \text{inv}(x) + (\text{inv}(x + 0.090373) * 3.8193e-09)$
11	1.561e-14	1.651e-01	$y = (\text{inv}(x * (0.090373 + x)) * -3.3928e-09) + \text{inv}(x)$
13	1.549e-14	4.069e-03	$y = ((\text{inv}((x + 0.090373) * x) + -2.9815) * -2.5837e-09...)$
14	1.276e-14	1.935e-01	$y = ((\text{inv}((x + 0.090373) * x) + \text{inv}(x)) * -2.5837e-09...)$
16	6.706e-15	3.218e-01	$y = ((\text{inv}(x + 1.6718) + \text{inv}(x * (x + 0.090373))) * -3...)$
18	6.519e-15	1.410e-02	$y = ((\text{inv}(x + (x + 3.3436)) + \text{inv}(x * (x + 0.090373)...)) * -2.9622e-09) + \text{inv}(x)$
19	3.819e-15	5.347e-01	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x) + \text{inv}((x + 0.090373) * x))) * -2.5837e-09) + \text{inv}(x)$
21	3.677e-15	1.896e-02	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x * 0.87396) + \text{inv}((0.090373 + x) * x))) * -2.5837e-09) + \text{inv}(x)$
23	3.553e-15	1.720e-02	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x * (1.5795 + x)) + \text{inv}((0.090373 + x) * x))) * -2.5837e-09) + \text{inv}(x)$
27	3.517e-15	2.513e-03	$y = \text{inv}(x) + ((\text{inv}(x + 1.6718) + (\text{inv}(x * (x + 0.090373)) + \text{inv}(x + ((x + x) * x) * x))) * -2.5782e-09...)$

)

Best equations for output 2

Complexity	Loss	Score	Equation
1	6.356e+02	0.000e+00	$y = -0.22241$
2	2.336e-12	3.324e+01	$y = \text{inv}(x)$
6	2.336e-12	2.131e-06	$y = \text{inv}(x) + (x * -4.92e-09)$
7	1.358e-14	5.148e+00	$y = \text{inv}(x) + (\text{inv}(x) * -4.0195e-08)$
9	7.609e-15	2.896e-01	$y = (\text{inv}(x + 0.0063144) * 1.4035e-08) + \text{inv}(x)$
11	7.598e-15	7.300e-04	$y = \text{inv}(x) + ((\text{inv}(x + 0.0039564) + -0.080935) * -2.68...52e-08)$
13	7.538e-15	3.961e-03	$y = \text{inv}(x) + (\text{inv}(x * (x + (x + 2.8532))) * -5.0997e...-09)$
14	5.336e-15	3.456e-01	$y = \text{inv}(x) + ((\text{inv}(x + 0.0047919) + \text{inv}(x + -0.21554))... * -7.432e-09)$
16	5.331e-15	4.163e-04	$y = \text{inv}(x) + (((\text{inv}(x + -0.21554) + 0.62131) + \text{inv}(x ... + 0.0047919)) * -7.432e-09)$
18	5.329e-15	2.083e-04	$y = \text{inv}(x) + (((\text{inv}(x + -0.21554) + (x * 0.24724)) + ... \text{inv}(x + 0.0047919)) * -7.432e-09)$
19	5.194e-15	2.575e-02	$y = \text{inv}(x) + (((\text{inv}(x + -0.21554) + \text{inv}(x + 0.0047919...)) + \text{inv}(x + 0.39774)) * -7.432e-09)$
20	4.670e-15	1.064e-01	$y = \text{inv}(x) + ((\text{inv}(x + -0.21554) + (\text{inv}(x + 0.0047919...)) + \text{inv}(x + \text{inv}(x)))) * -7.432e-09)$
24	4.128e-15	3.083e-02	$y = (\text{inv}((x + x) + (((\text{inv}((x + 0.97401) + x) + 3.559...8) * (x * x)) + 2.8532)) * -7.2074e-08) + \text{inv}(x)$
25	3.062e-15	2.987e-01	$y = \text{inv}(x) + (\text{inv}((((x * ((\text{inv}(\text{inv}(x + -3.5763) + x))... + 4.7066) * x)) + x) + x) + 2.8532) * -7.2074e-08)$

Best equations for output 3

Complexity	Loss	Score	Equation
1	1.910e+04	0.000e+00	$y = 13.156$
2	1.490e-10	3.248e+01	$y = \text{inv}(x)$
4	6.788e-15	4.998e+00	$y = \text{inv}(x + 4.7591e-11)$

```

8          6.786e-15  8.179e-05  y = inv(x + 3.1218e-11) + (x * 2.7958e-09)
9          6.779e-15  9.821e-04  y = inv(x + 8.5559e-11) + (inv(x) * 8.5559e-
11)
11         2.809e-15  4.405e-01  y = inv(x + 7.9835e-11) + (inv(x + -
0.19597) * 2.66e-0...
                                     8)
13         2.098e-15  1.458e-01  y = inv(x + 4.6222e-11) + (inv((x + -
0.39619) + x) * ...
                                     4.5114e-08)

```

Press 'q' and then <enter> to stop execution early.

Complexity	Loss	Score	Equation
1	4.969e+01	0.000e+00	$y = -1.3212$
2	3.991e-14	3.476e+01	$y = \text{inv}(x)$
6	3.991e-14	5.563e-05	$y = (x * 4.9903e-09) + \text{inv}(x)$
7	3.139e-14	2.401e-01	$y = \text{inv}((\text{inv}(x) * 2.2184e-10) + x)$
9	2.172e-14	1.840e-01	$y = \text{inv}(x) + (\text{inv}(x + 0.090373) * 3.8193e-09)$
11	1.561e-14	1.651e-01	$y = (\text{inv}(x * (0.090373 + x)) * -3.3928e-09) + \text{inv}(x)$
13	1.549e-14	4.069e-03	$y = ((\text{inv}((x + 0.090373) * x) + -2.9815) * -2.5837e-09...)$
14	1.276e-14	1.935e-01	$y = ((\text{inv}((x + 0.090373) * x) + \text{inv}(x)) * -2.5837e-09...)$
16	6.528e-15	3.352e-01	$y = ((\text{inv}((x + 0.090373) * x) + \text{inv}(x + 1.6718)) * -2...)$
18	6.519e-15	6.807e-04	$y = ((\text{inv}(x + (x + 3.3436)) + \text{inv}(x * (x + 0.090373)...)) * -2.9622e-09) + \text{inv}(x)$
19	3.819e-15	5.347e-01	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x) + \text{inv}((x + 0.090373) * x))) * -2.5837e-09) + \text{inv}(x)$
21	3.677e-15	1.896e-02	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x * 0.87396) + \text{inv}((0.090...373 + x) * x))) * -2.5837e-09) + \text{inv}(x)$
23	3.553e-15	1.720e-02	$y = ((\text{inv}(x + 1.6718) + (\text{inv}(x * (1.5795 + x)) + \text{inv}(...(0.090373 + x) * x))) * -2.5837e-09) + \text{inv}(x)$
27	3.517e-15	2.513e-03	$y = \text{inv}(x) + ((\text{inv}(x + 1.6718) + (\text{inv}(x * (x + 0.090...373)) + \text{inv}(x + ((x + x) * x) * x))) * -2.5782e-09...$

)

Complexity	Loss	Score	Equation
1	6.356e+02	0.000e+00	$y = -0.22241$
2	2.336e-12	3.324e+01	$y = \text{inv}(x)$
6	2.336e-12	2.131e-06	$y = \text{inv}(x) + (x * -4.92e-09)$
7	1.358e-14	5.148e+00	$y = \text{inv}(x) + (\text{inv}(x) * -4.0195e-08)$
9	7.609e-15	2.896e-01	$y = (\text{inv}(x + 0.0063144) * 1.4035e-08) + \text{inv}(x)$
11	7.598e-15	7.300e-04	$y = \text{inv}(x) + ((\text{inv}(x + 0.0039564) + -$ 0.080935) * -2.68... 52e-08)
13	6.783e-15	5.672e-02	$y = \text{inv}(x) + (\text{inv}((x + -1.2705) * (0.0047919 + x)) * ...$ -7.432e-09)
14	5.336e-15	2.401e-01	$y = \text{inv}(x) + ((\text{inv}(x + 0.0047919) + \text{inv}(x + -$ 0.21554) ...) * -7.432e-09)
16	5.325e-15	1.041e-03	$y = \text{inv}(x) + ((\text{inv}(x + -0.21075) + (\text{inv}(x + 0.0047919...)$) + -1.1023)) * -7.432e-09)
19	5.194e-15	8.304e-03	$y = \text{inv}(x) + (((\text{inv}(x + -0.21554) + \text{inv}(x + 0.0047919...)$) + \text{inv}(x + 0.39774)) * -7.432e-09)
20	4.670e-15	1.064e-01	$y = \text{inv}(x) + ((\text{inv}(x + -0.21554) + (\text{inv}(x + 0.0047919...)$) + \text{inv}(x + \text{inv}(x)))) * -7.432e-09)
24	4.128e-15	3.083e-02	$y = (\text{inv}((x + x) + (((\text{inv}((x + 0.97401) + x) + 3.559...)$ 8) * (x * x)) + 2.8532)) * -7.2074e- 08) + \text{inv}(x)
25	3.062e-15	2.987e-01	$y = \text{inv}(x) + (\text{inv}((((x * ((\text{inv}(\text{inv}(x + -$ 3.5763) + x) ... + 4.7066) * x)) + x) + x) + 2.8532) * - 7.2074e-08)

Complexity	Loss	Score	Equation
1	1.910e+04	0.000e+00	$y = 13.156$
2	1.490e-10	3.248e+01	$y = \text{inv}(x)$
4	6.788e-15	4.998e+00	$y = \text{inv}(x + 4.7591e-11)$
8	6.786e-15	8.179e-05	$y = \text{inv}(x + 3.1218e-11) + (x * 2.7958e-09)$
9	6.779e-15	9.821e-04	$y = \text{inv}(x + 8.5559e-11) + (\text{inv}(x) * 8.5559e-$ 11)
11	2.667e-15	4.665e-01	$y = \text{inv}(x + 8.6609e-11) + (\text{inv}(x + -$ 0.22145) * 2.131e-... 08)
13	2.098e-15	1.199e-01	$y = \text{inv}(x + 4.6222e-11) + (\text{inv}((x + -$

0.39619) + x) * ...

4.5114e-08)

- outputs\20260509_142127_3XbnpJ\hall_of_fame_output1.csv
- outputs\20260509_142127_3XbnpJ\hall_of_fame_output2.csv
- outputs\20260509_142127_3XbnpJ\hall_of_fame_output3.csv

[Info: Final populations:

[Info: Output 1:

[Info: Output 2:

[Info: Output 3:

[Info: Results saved to:

model_selection	'best'
binary_operators	['+', '*']
unary_operators	['inv(x) = 1/x']
expression_spec	None
niterations	100
populations	31
population_size	27
max_evals	None
maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None
nested_constraints	None
elementwise_loss	None
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None
parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True
use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0

alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036
fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502
weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12
should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None
optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None
heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False
turbo	False
bumper	False

precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False
verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None
delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	{'inv': <function <la...001A2520607C0>>}
extra_torch_mappings	None
extra_jax_mappings	None
denoise	False
select_k_features	None

model

model_selection	'best'
binary_operators	['+', '*']
unary_operators	['inv(x) = 1/x']
expression_spec	None
niterations	100
populations	31
population_size	27
max_evals	None
maxsize	30
maxdepth	None
warmup_maxsize_by	None
timeout_in_seconds	None
constraints	None

nested_constraints	None
elementwise_loss	None
loss_function	None
loss_function_expression	None
loss_scale	'log'
complexity_of_operators	None
complexity_of_constants	None
complexity_of_variables	None
complexity_mapping	None
parsimony	0.0
dimensional_constraint_penalty	None
dimensionless_constants_only	False
use_frequency	True
use_frequency_in_tournament	True
adaptive_parsimony_scaling	1040.0
alpha	3.17
annealing	False
early_stop_condition	None
ncycles_per_iteration	380
fraction_replaced	0.00036
fraction_replaced_hof	0.0614
weight_add_node	2.47
weight_insert_node	0.0112
weight_delete_node	0.87
weight_do_nothing	0.273
weight_mutate_constant	0.0346
weight_mutate_operator	0.293
weight_swap_operands	0.198
weight_rotate_tree	4.26
weight_randomize	0.000502
weight_simplify	0.00209
weight_optimize	0.0
crossover_probability	0.0259
skip_mutation_failures	True
migration	True
hof_migration	True
topn	12
should_simplify	True
should_optimize_constants	True
optimizer_algorithm	'BFGS'
optimizer_nrestarts	2
optimizer_f_calls_limit	None

optimize_probability	0.14
optimizer_iterations	8
perturbation_factor	0.129
probability_negate_constant	0.00743
tournament_selection_n	15
tournament_selection_p	0.982
parallelism	None
procs	None
cluster_manager	None
heap_size_hint_in_bytes	None
batching	False
batch_size	50
fast_cycle	False
turbo	False
bumper	False
precision	32
autodiff_backend	None
random_state	None
deterministic	False
warm_start	False
verbosity	1
update_verbosity	None
print_precision	5
progress	True
logger_spec	None
input_stream	'stdin'
run_id	None
output_directory	None
temp_equation_file	False
tempdir	None
delete_tempfiles	True
update	False
output_jax_format	False
output_torch_format	False
extra_sympy_mappings	{'inv': <function <la...001A2520607C0>>}
extra_torch_mappings	None
extra_jax_mappings	None
denoise	False
select_k_features	None

Julia packages and types

PySR uses [SymbolicRegression.jl](#) as its search backend. This is a pure Julia package, and so can interface easily with any other Julia package. For some tasks, it may be necessary to load such a package.

For example, let's say we wish to discovery the following relationship:

$$y = p_{3x+1} - 5,$$

where p_i is the i th prime number, and x is the input feature.

Let's see if we can discover this using the [Primes.jl](#) package.

`jl` stores the Julia runtime.

Now, let's run some Julia code to add the `Primes.jl` package to the PySR environment:

```
from pysr import jl

jl.seval(
    """
import Pkg
Pkg.add("Primes")
"""
)
```

This imports the Julia package manager, and uses it to install `Primes.jl`. Now let's import `Primes.jl`:

```
jl.seval("using Primes: prime")
```

Note that PySR should automatically load the `juliacall.ipython` extension for you, which means that you can also execute Julia code in the notebook using the `%julia` magic:

```
%julia using Primes: prime
```

Now, we define a custom operator:

```

jl.seval(
    """
function p(i::T) where T
    if 0.5 < i < 1000
        return T(prime(round(Int, i)))
    else
        return T(NaN)
    end
end
"""
)

```

Or, equivalently:

```

%%julia
function p(i::T) where T
    if 0.5 < i < 1000
        return T(prime(round(Int, i)))
    else
        return T(NaN)
    end
end
end

```

We have created a function `p`, which takes a number `i` of type `T` (e.g., `T=Float64`). `p` first checks whether the input is between 0.5 and 1000. If out-of-bounds, it returns `NaN`. If in-bounds, it rounds it to the nearest integer, computes the corresponding prime number, and then converts it to the same type as input.

The equivalent function in Python would be:

```

import sympy

def p(i):
    if 0.5 < i < 1000:
        return float(sympy.prime(int(round(i))))
    else:
        return float("nan")

```

(However, note that this version assumes 64-bit float input, rather than any input type `T`)

Next, let's generate a list of primes for our test dataset. Since we are using `juliacall`, we can just call `p` directly to do this:

```
primes = {i: j1.p(i * 1.0) for i in range(1, 999)}
```

Next, let's use this list of primes to create a dataset of x, y pairs:

```
import numpy as np

X = np.random.randint(0, 100, 100)[: , None]
y = [primes[3 * X[i, 0] + 1] - 5 + np.random.randn() * 0.001 for i in range(100)]
```

Note that we have also added a tiny bit of noise to the dataset.

Finally, let's create a PySR model, and pass the custom operator. We also need to define the sympy equivalent, which we can leave as a placeholder for now:

```
from pysr import PySRRegressor
import sympy

class sympy_p(sympy.Function):
    pass

model = PySRRegressor(
    binary_operators=["+", "-", "*", "/"],
    unary_operators=["p"],
    niterations=20,
    extra_sympy_mappings={"p": sympy_p},
)
```

We are all set to go! Let's see if we can find the true relation:

```
model.fit(X, y)
```

if all works out, you should be able to see the true relation (note that the constant offset might not be exactly 1, since it is allowed to round to the nearest integer).

You can get the sympy version of the best equation with:

```
model.sympy()
```

High-dimensional input: Neural Nets + Symbolic Regression

In this example, let's learn a high-dimensional problem. **This will use the method proposed in our NeurIPS paper: <https://arxiv.org/abs/2006.11287>.**

Let's consider a time series problem:

$$z = y^2, \quad y = \frac{1}{10} \sum (y_i), \quad y_i = x_{i0}^2 + 6 \cos(2 * x_{i2})$$

Imagine our time series is 10 timesteps. That is very hard for symbolic regression, even if we impose the inductive bias of

$$z = f\left(\sum g(x_i)\right)$$

- it is the square of the number of possible equations!

But, as in our paper, **we can break this problem down into parts with a neural network. Then approximate the neural network with the symbolic regression!**

Then, instead of, say, $(10^9)^2 = 10^{18}$ equations, we only have to consider 2×10^9 equations.

```
import numpy as np

rstate = np.random.RandomState(0)

N = 100000
Nt = 10
X = 6 * rstate.rand(N, Nt, 5) - 3
y_i = X[..., 0] ** 2 + 6 * np.cos(2 * X[..., 2])
y = np.sum(y_i, axis=1) / y_i.shape[1]
z = y**2
X.shape, y.shape
```

Neural Network definition

So, as described above, let's first use a neural network with the sum inductive bias to solve this problem.

Essentially, we will learn two neural networks: - **f** - **g**

each defined as a multi-layer perceptron. We will sum over **g** the same way as in our equation, but we won't define the summed part beforehand.

Then, we will fit **g** and **f** **separately** using symbolic regression.

Warning

We import torch *after* already starting PyJulia. This is required due to interference between their C bindings. If you use torch, and then run PyJulia, you will likely hit a segfault. So keep this in mind for mixed deep learning + PyJulia/PySR workflows.

```
!pip install pytorch_lightning
```

```
import torch
from torch import nn, optim
from torch.nn import functional as F
from torch.utils.data import DataLoader, TensorDataset
import pytorch_lightning as pl

hidden = 128
total_steps = 50_000

def mlp(size_in, size_out, act=nn.ReLU):
    return nn.Sequential(
        nn.Linear(size_in, hidden),
        act(),
        nn.Linear(hidden, hidden),
        act(),
        nn.Linear(hidden, hidden),
        act(),
        nn.Linear(hidden, size_out),
    )

class SumNet(pl.LightningModule):
    def __init__(self):
        super().__init__()

        #####
        # The same inductive bias as above!
        self.g = mlp(5, 1)
        self.f = mlp(1, 1)

    def forward(self, x):
        y_i = self.g(x)[: , : , 0]
        y = torch.sum(y_i, dim=1, keepdim=True) / y_i.shape[1]
```

```

        z = self.f(y)
        return z[:, 0]

#####

# PyTorch Lightning bookkeeping:
def training_step(self, batch, batch_idx):
    x, z = batch
    predicted_z = self(x)
    loss = F.mse_loss(predicted_z, z)
    return loss

def validation_step(self, batch, batch_idx):
    return self.training_step(batch, batch_idx)

def configure_optimizers(self):
    optimizer = torch.optim.Adam(self.parameters(), lr=self.max_lr)
    scheduler = {
        "scheduler": torch.optim.lr_scheduler.OneCycleLR(
            optimizer,
            max_lr=self.max_lr,
            total_steps=self.trainer.estimated_stepping_batches,
            final_div_factor=1e4,
        ),
        "interval": "step",
    }
    return [optimizer], [scheduler]

```

Data bookkeeping

Put everything into PyTorch and do a train/test split:

```

from multiprocessing import cpu_count

Xt = torch.tensor(X).float()
zt = torch.tensor(z).float()
X_train, X_test, z_train, z_test = train_test_split(Xt, zt, random_state=0)
train_set = TensorDataset(X_train, z_train)
train = DataLoader(
    train_set, batch_size=128, num_workers=cpu_count(), shuffle=True, pin_memory=True
)

```

```
test_set = TensorDataset(X_test, z_test)
test = DataLoader(test_set, batch_size=256, num_workers=cpu_count(), pin_memory=True)
```

Train the model with PyTorch Lightning on GPUs:

Start the model:

```
pl.seed_everything(0)
model = SumNet()
model.total_steps = total_steps
model.max_lr = 1e-2
```

PyTorch Lightning trainer object:

```
trainer = pl.Trainer(max_steps=total_steps, accelerator="gpu", devices=1)
```

Here, we fit the neural network:

```
trainer.fit(model, train_dataloaders=train, val_dataloaders=test)
```

Latent vectors of network

Let's get the input and output of the learned g function from the network over some random data:

```
np.random.seed(0)
idx = np.random.randint(0, 10000, size=1000)

X_for_pysr = Xt[idx]
y_i_for_pysr = model.g(X_for_pysr)[: , : , 0]
y_for_pysr = torch.sum(y_i_for_pysr, dim=1) / y_i_for_pysr.shape[1]
z_for_pysr = zt[idx] # Use true values.

X_for_pysr.shape, y_i_for_pysr.shape
```


Learning over the network:

Now, let's fit g using PySR.

Warning

First, let's save the data, because sometimes PyTorch and juliacall's C bindings interfere and cause the colab kernel to crash. If we need to restart, we can just load the data without having to retrain the network:

```
nnet_recordings = {
    "g_input": X_for_pysr.detach().cpu().numpy().reshape(-1, 5),
    "g_output": y_i_for_pysr.detach().cpu().numpy().reshape(-1),
    "f_input": y_for_pysr.detach().cpu().numpy().reshape(-1, 1),
    "f_output": z_for_pysr.detach().cpu().numpy().reshape(-1),
}

# Save the data for later use:
import pickle as pkl

with open("nnet_recordings.pkl", "wb") as f:
    pkl.dump(nnet_recordings, f)
```

We can now load the data, including after a crash (be sure to re-run the import cells at the top of this notebook, including the one that starts juliacall).

```
import pickle as pkl

nnet_recordings = pkl.load(open("nnet_recordings.pkl", "rb"))
f_input = nnet_recordings["f_input"]
f_output = nnet_recordings["f_output"]
g_input = nnet_recordings["g_input"]
g_output = nnet_recordings["g_output"]
```

And now fit using a subsample of the data (symbolic regression only needs a small sample to find the best equation):

```
rstate = np.random.RandomState(0)
f_sample_idx = rstate.choice(f_input.shape[0], size=500, replace=False)

model = PySRRegressor(
    niterations=50,
```

```

    binary_operators=["+", "-", "*"],
    unary_operators=["cos", "square"],
)
model.fit(g_input[f_sample_idx], g_output[f_sample_idx])

```

If this segfaults, restart the notebook, and run the initial imports and juliacall part, but skip the PyTorch training. This is because PyTorch's C binding tends to interfere with juliacall. You can then re-run the `pkl.load` cell to import the data.

Validation

Recall we are searching for f and g such that:

$$z = f\left(\sum g(x_i)\right)$$

which approximates the true relation:

$$z = y^2, \quad y = \frac{1}{10} \sum (y_i), \quad y_i = x_{i0}^2 + 6 \cos(2x_{i2})$$

Let's see how well we did in recovering g :

```

model.equations_[["complexity", "loss", "equation"]]

```

A neural network can easily undo a linear transform (which commutes with the summation), so any affine transform in g is to be expected. The network for f has learned to undo the linear transform.

This likely won't find the exact result, but it should find something similar. You may wish to try again but with many more `total_steps` for the neural network (10,000 is quite small!).

Then, we can learn another analytic equation for f .

Now, we can compose these together to get the time series model!

Think about what we just did: we found an analytical equation for z in terms of 500 datapoints, under the assumption that z is a function of a sum of another function over an axis:

$$z = f\left(\sum_i g(x_i)\right)$$

And we pulled out analytical copies for g using symbolic regression.

Other PySR Options

The full list of PySR parameters can be found here: <https://astroautomata.com/PySR/api>